

1 Sorting and searching**1.1 Insertion sort**

Take elements from right to left, insert them in the right sorted portion of the list. This algorithm sorts **in-place**.

Time complexity:

- Best: $\Theta(n)$; already sorted array
- Average: $\Theta(n^2)$
- Worst: $\Theta(n^2)$; array sorted in reverse

Space complexity: Recursion depth = n ; re-uses the tail list.

- Best / Average / Worst: $\Theta(n)$

```
1 function insert(x: number, xs: List<number>): List<number> {
2   if (is_null(xs)) {
3     return list(xs);
4   } else if (x <= head(xs)) {
5     // found correct position, resolve
6     return pair(x, xs);
7   } else {
8     // move element ahead
9     return pair(head(xs), insert(x, tail(xs)));
10  }
11 }
12
13 function insertion_sort(xs: List<number>): List<number> {
14   return is_null(xs)
15     ? xs
16     : insert(head(xs), insertion_sort(tail(xs)));
17 }
```

1.2 Selection sort

Choose the smallest element in a list. Place that element in order in a new list.

Time complexity: Number of comparisons is fixed.

- Best/Average/Worst: $\Theta(n^2)$

Space complexity: Recursion depth = n ; remove() creates an n sized copy.

- Best/Average/Worst: $\Theta(n^2)$

```
1 function smallest(xs: List<number>): number {
2   function h(xs: List<number>, min: number): number {
3     return is_null(xs)
4       ? reached the end
5       : min
6       : head(xs) < min
7         ? found a smaller number than min. use that
8         : h(tail(xs), head(xs))
9       : h(tail(xs), min);
10  }
11  return h(xs, head(xs));
12 }
13
14 function selection_sort(xs: List<number>): List<number> {
15   if (is_null(xs)) {
16     return xs;
17   } else {
18     const s = smallest(xs);
19     // smallest elem s goes in front of the list
20     return pair(s, selection_sort(remove(s, xs)));
21   }
22 }
```

1.3 Quick sort

A divide-and-conquer algorithm. Choose a pivot, and partition the remaining elements into 2 groups: smaller and larger. Then recursively partition the groups: quicksort(smaller) + [pivot] + quicksort(larger).

Time complexity:

- Best: $\Theta(n \log n)$
- Average: $\Theta(n \log n)$
- Worst: $\Theta(n^2)$; pivot always unbalanced, ie. smaller $\ll$ bigger or bigger $\gg$ smaller

Space complexity:

- Best: $\Theta(n \log n)$
- Average: $\Theta(n \log n)$; recursion depth = $\log n$.
- Worst: $\Theta(n^2)$; pivot always unbalanced, recursion depth = n .

```
1 function partition(xs: List<number>, pivot: number): Pair<List<number>,
2   List<number>> {
3   return accumulate(
4     (curr, acc) => {
5       if (curr <= xs) {
6         return pair(
7           pair(curr, head(acc)),
8           tail(acc)
9         );
10      } else {
11        return pair(
12          head(acc),
13          pair(curr, tail(acc))
14        );
15      }
16    }, pair(list(), list()), // 0 is smaller, 1 is larger
17    xs
18  );
19 }
20
21 function quicksort(xs: List<number>): List<number> {
22   if (is_null(xs)) {
23     return list(null);
24   } else {
25     const pivot = head(xs);
26     const s = partition(tail(xs), pivot);
```

```
27     return append(
28       quicksort(head(s)), // smaller
29       append(
30         list(pivot),
31         quicksort(tail(s)) // larger
32       );
33     );
34   }
35 }
```

1.4 Merge sort

Divide and conquer. Recursively split unsorted elements and then merge them back together sorted.

Note: see utilities for implementation of take() and drop().

Time complexity: Splitting creates $\log n$ problems of merging lists of size n

- Best/Average/Worst: $\Theta(n \log n)$

Space complexity: Merging creates a list with total size n

- Best/Average/Worst: $\Theta(n)$

```
1 function merge(xs: List<number>, ys: List<number>): List<number> {
2   // base cases: run out of elements in one of the lists, just append the
3   other list
4   if (is_null(xs)) {
5     return ys;
6   } else if (is_null(ys)) {
7     return xs;
8   }
9
10  function merge_sort(xs: List<number>): List<number> {
11    if (is_null(xs) || is_null(tail(xs))) {
12      return xs;
13    } else {
14      const mid = math_floor(length(xs) / 2);
15      return merge(
16        // continue splitting until base case
17        merge_sort(take(xs, mid)),
18        merge_sort(drop(xs, mid))
19      );
20    }
21  }
```

1.5 Binary search

Assume sorted list.

Time complexity: list_ref() is an $O(n)$ operation.

- Best: $\Theta(n)$; target in the middle
- Average: $\Theta(n)$
- Worst: $\Theta(n)$; $\log n$ iterations

Space complexity: $\log n$ iterations asymptotically.

- Best/Average/Worst: $\Theta(\log n)$

```
1 function binary_search(xs, target) {
2   function helper(low, high) {
3     if (low > high) {
4       return -1;
5     } else {
6       const mid = math_floor((low + high) / 2);
7       const v = list_ref(xs, mid);
8
9       if (v === target) {
10        return mid;
11      } else if (target < v) {
12        return helper(low, mid - 1); // search left
13      } else {
14        return helper(mid + 1, high); // search right
15      }
16    }
17    return helper(0, length(xs) - 1);
18  }
```

2 Permutations and Combinations**2.1 Permutations**

Get permutations of a list. Order matters.

```
1 function permutations<T>(xs: List<T>): List<List<T>> {
2   if (is_null(xs)) {
3     return list(null);
4   } else {
5     // Flatten list [[1,2]], [[2, 1]] => [[1,2], [2, 1]]
6     // x is prepended to all possible permutations
7     return accumulate(
8       append(
9         list(),
10        map(
11          // choose x from xs
12          x => map(
13            // prepend x to the permutations of remaining
14            p => pair(x, p),
15            // recursively generate all permutations of remaining list
16            permutations(remove(x, xs))
17          ),
18          xs
19        );
20      );
21    }
22  }
```

2.2 Combinations

Get combinations of sub-lists (length r) of a list.

```
1 // 0 <= r <= n
```

```
2 function combinations<T>(xs: List<T>, r: number): List<List<T>> {
3   if (r === 0) {
4     return list(null);
5   } else if (is_null(xs)) {
6     return null;
7   } else {
8     // we do not choose the head(xs)
9     const no = combinations(tail(xs), r);
10    // choose the head(xs) and prepend it to possible combinations of
11    length r - 1
12    const yes = map(
13      x => pair(head(xs), x),
14      combinations(tail(xs), r - 1)
15    );
16    return append(no, yes);
17  }
```

2.3 N Choose R

```
1 // Formula: nCr = n! / (r! * (n-r)!)
2 // Recursive: nCr = (n-1)C(r-1) + (n-1)C(r) (Pascal's triangle)
3 // Example: 5C3 = 4C2 + 4C3 = 6 + 4 = 10
4 function nCr(n, r) {
5   if (r > n || r < 0) return 0;
6   if (r === 0) return 1;
7
8   // choose + dont choose
9   return nCr(n - 1, r - 1) + nCr(n - 1, r);
10 }
```

2.4 N Permute R

```
1 // Formula: nPr = n! / (n-r)!
2 // Recursive: nPr = n * (n-1)P(r-1)
3 // Example: 5P3 = 5 * 4P2 = 5 * 4 * 3P1 = 5 * 4 * 3 = 60
4 function nPr(n, r) {
5   if (r > n || r < 0) return 0;
6   if (r === 0) return 1;
7
8   // prepnd all head elems onto the remainder
9   x => pair(head(xs), x),
10  subsets(tail(xs))
11 );
12 return append(no, yes);
13 }
```

2.5 Subsets

Get all possible subsets of xs. Modified version of combinations.

```
1 function subsets<T>(xs: List<T>): List<List<T>> {
2   if (is_null(xs)) {
3     return list(null);
4   } else {
5     const no = subsets(tail(xs));
6     const yes = map(
7       // prepend all head elems onto the remainder
8       x => pair(head(xs), x),
9       subsets(tail(xs))
10     );
11     return append(no, yes);
12   }
13 }
```

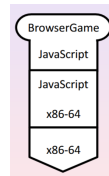
3 Tombstone diagrams**3.1 Interpreter**

Figure 1: Interpreting JS to x86 machine code

3.2 Compiler

T-figure: Language 1 (left) compiles to language 2 (right) via compiler written with language 3 (bottom).

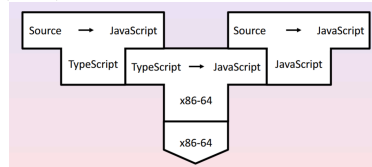


Figure 2: Compiling a compiler

4 Space complexity, Time complexity**4.1 Order of growth**

Big Theta: The function r has order of growth $\Theta(g(n))$ if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \cdot g(n) \leq r(n) \leq k_2 \cdot g(n)$ for any $n > n_0$.

Big O: The function r has order of growth $O(g(n))$ if there is a positive constant k such that $r(n) \leq k \cdot g(n)$ for any sufficiently large value of n .

Big Omega: The function r has order of growth $\Omega(g(n))$ if there is a positive constant k such that $k \cdot g(n) \leq r(n)$ for any sufficiently large value of n .

Order (small to big): 1, $\log n$, n , $n \log n$, n^k , k^n , n^n

4.2 Tips for analysis

Recurrence Notation: $T(n)$ represents the time complexity for input size n . The $+$ separates the work done at the current level from the recursive calls. For example, $T(n) = \Theta(1) + 2T(n/2)$ means constant work at this level, plus two recursive calls on half-sized problems. The coefficient before T indicates the number of recursive calls (e.g., $kT(n-1)$ means k recursive calls).

Case	Recurrence / Pattern	Complexity
Linear scan	$T(n) = \Theta(1) + T(n-1)$	$\Theta(n)$
Nested loops (k loops)	-	$\Theta(n^k)$
Divide & conquer (typical)	$T(n) = \Theta(n) + 2T(n/2)$	$\Theta(n \log n)$
Subproblems (k per step)	$T(n) = kT(n-1)$	$\Theta(k^n)$
Recursive calls to $f(n-k)$	$T(n) = \Theta(1) + T(n-a)$	$\Theta(n)$
Recursive calls to $f(n-k)$ with linear work	$T(n) = \Theta(n) + T(n-a)$	$\Theta(n^2)$
Recursive calls to $f(n/k)$	$T(n) = \Theta(1) + T(n/a)$	$\Theta(\log n)$
Recursive calls to $f(n/k)$ with linear work	$T(n) = \Theta(n) + T(n/a)$	$\Theta(n)$
Branching recursion (b subproblems of size $n-a$)	$T(n) = \Theta(1) + bT(n-a)$	$\Theta(c^n)$
Branching recursion with linear work	$T(n) = \Theta(n) + bT(n-a)$	$\Theta(c^n)$
Divide & conquer with constant work	$T(n) = \Theta(1) + aT(n/a)$	$\Theta(n)$
Divide & conquer with linear work	$T(n) = \Theta(n) + aT(n/a)$	$\Theta(n \log n)$
Work decreases per recursive call	$T(n) = g(i)$ per level, depth = d	$\Theta(\sum_{i=0}^d g(i))$

5 Utilities**5.1 Definitions**

Tree: A tree of a certain data type is a list whose elements are of that data type, or trees of that data type.

```
1 type Tree<T> = List<T | Tree<T> | null>
```

List: A list is either null or a pair whose tail is a list.

5.2 Take/Drop**5.2.1 Take**

Take the first n elements.

```
1 function take<T>(xs: List<T>, n: number): List<T> {
2   return n === 0
3     ? null
4     : pair(head(xs), take(tail(xs), n - 1));
5 }
```

5.2.2 Drop

Drop the first n elements.

```
1 function drop<T>(xs: List<T>, n: number): List<T> {
2   return n === 0
3     ? xs
4     : drop(tail(xs), n - 1);
5 }
```

5.3 Unique

Get all unique elements.

```

1 function unique<T>(xs: List<T>): List<T> {
2   return is_null(xs)
3     ? null
4     : append(
5       list(head(xs)),
6       unique(remove_all(head(xs), tail(xs)))
7     );
8 }

```

5.4 Continuation-Passing Style

Motivation: A programming pattern where we convert recursive functions into iterative ones by passing the “rest of the computation” as a function argument. This deferred operation (the continuation) accumulates the work to be done after the current step completes. CPS can convert any recursive function into an iterative form.

Recursive version:

```

1 function factorial(n: number): number {
2   return n == 1
3     ? 1
4     : n * factorial(n - 1);
5 }

```

CPS version (iterative):

```

1 function fac<T>(n: number, c: (x: number) => T): T {
2   return n == 1
3     ? c(1)
4     : fac(n - 1, x => c(n * x));
5 }
6
7 function factorial_iter(n: number): number {
8   return fac(n, x => x);
9 }

```

The continuation `c` accumulates the pending multiplications. Instead of building up a call stack, we build up a chain of functions that will be applied once we reach the base case.

5.5 CSE

5.6 Control

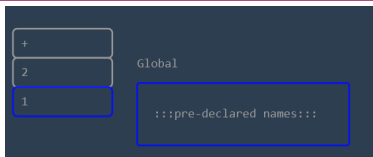


Figure 3: Evaluating `1+2`:: LHS pushed onto control before RHS, stash takes values from bottom of control

5.7 Environment

5.7.1 Assignments

- `.` for let
- `:=` for const and function

5.7.2 Functions

- Left ball points towards frame
- Right ball contains closure with:
 1. parameters
 2. function body

5.8 Scoping

- New frames only created if
 1. block scope has declarations
 2. function call has parameters
- function calls travel up the tree to find a declaration